

Teaching a Neural Network to Predict the Cost of the Future, for Faster MPC

Alessandro Wegher

github.com/alewegher/ORC_with_FP — full technical report inside the repo

The problem. Model Predictive Control (MPC) re-plans a robot's motion at every time step by solving an optimization problem over a short future window. Look further ahead and the controller behaves more intelligently, but the optimization gets far more expensive to solve in real time. Look only a little ahead, and the controller becomes short-sighted: locally reasonable decisions, globally poor outcomes.

The idea. Instead of looking far ahead at every step, train a small neural network offline to predict “how expensive is it to finish well from here?” – the cost-to-go. Bolt that prediction onto a short-horizon controller as a *terminal cost*, and it gets a cheap, learned sense of the long-term consequences of its short-term choices, without paying for a long-horizon solve online.

Testbed. A single pendulum (1 joint) and a double pendulum (2 joints), both fully actuated. For each, three controllers are compared: a short-horizon MPC, a long-horizon MPC (expensive, near-optimal, the reference), and the same short horizon with the learned terminal cost added.

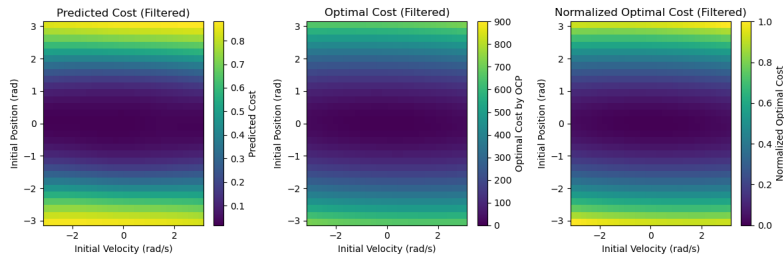


Figure 1: The network learns to predict cost-to-go directly from a state $(q, \dot{q})$ – brighter means a more expensive future. Left: network prediction. Middle: ground truth from solving the real optimization problem. Right: ground truth, normalized. The match is close almost everywhere.

Does it actually work in closed loop? Predicting the right cost on average across many starting points is one thing; actually steering a real, running controller the same way a long-horizon controller would is another. So beyond the aggregate comparison, each controller was run for real, in receding-horizon closed loop, from the same starting state:

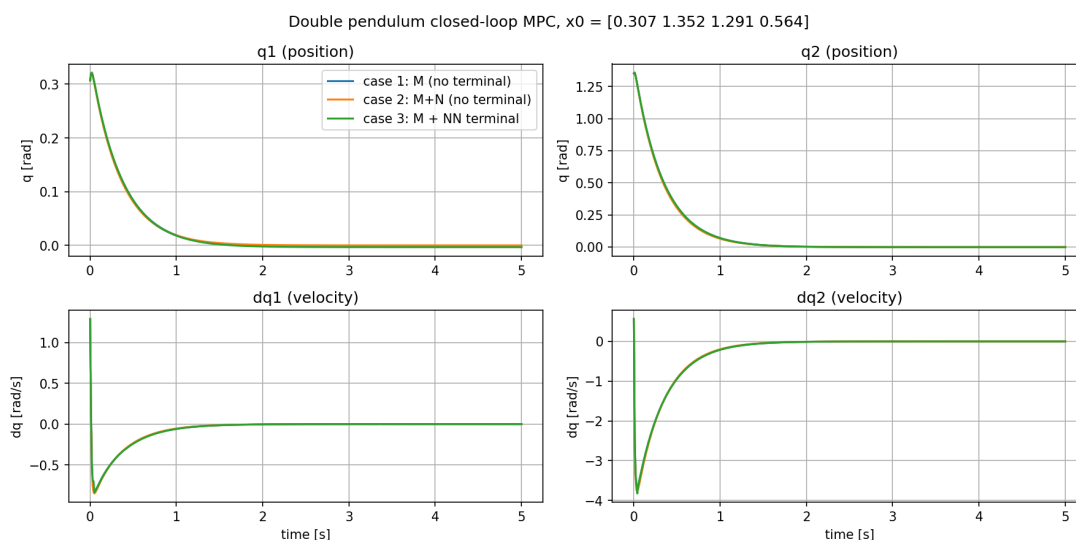


Figure 2: Double pendulum, closed loop: joint angles and velocities under the three controllers, all three curves essentially on top of each other. The short-horizon controller with the learned terminal cost matches the expensive long-horizon controller almost exactly – **at $3.5\times$ less compute** (51.7s vs. 178.9s of solver time for this run).

Takeaway. A cheap, learned terminal cost can recover the benefit of an expensive long-horizon MPC – as the double pendulum result above shows, at a fraction of the compute. But not for free, and not everywhere: on the simpler single-pendulum system, the same learned terminal cost sometimes fails to pull the short-horizon controller away from its myopic behaviour, even though it predicts the right cost *on average* across many starting states. That gap between “predicts the right value” and “actually steers the controller the right way” is, to me, the most interesting result here, and a natural next step (matching the value function’s *gradient*, not just its value, during training).

Full write-up, all code, and every figure: github.com/alewegher/ORC_with_FP